

30-Day Algorithm Whiteboard Sprint Plan

Tailored for MLSys PhDs & Low-Level Hardware Systems Mapping

Phase 1: Foundations & Pointer Intuition (Days 1-7)

Phase Target: Master two pointers (left-right, fast-slow), sliding windows, monotonic stacks/queues, and prefix sums. Build strong memory muscle for array/list transformations and eliminate boundary pointer bugs.

Day 01

Arrays & Two Pointers

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
1	Two Sum	Easy	Key Points: Hash Map / Two Pointers ⚡ MLSys Connection: Utilizing static hash maps or sorted two-pointer matching is the standard approach for fast table lookups in low-level systems and GPU kernel designs. Optimizing from $O(N^2)$ to $O(N)$ is a classic space-time tradeoff example.
15	3Sum	Medium	Key Points: Sorting + Left-Right Pointers ⚡ MLSys Connection: The core of two-pointer searches is search space reduction using monotonicity. Sorting allows us to shift pointers inwards conditionally, pruning the brute-force $O(N^3)$ search space to $O(N^2)$.
11	Container With Most Water	Medium	Key Points: Two Pointers + Greedy Shifting

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			⚡ MLSys Connection: Greedy algorithms are crucial in system scheduling (e.g. choosing nodes with the lowest network latency). Moving the pointer pointing to the shorter line is the only action that can potentially yield a larger area.

Day 02

Sliding Window

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			Key Points: Sliding Window + Hash Set
3	Longest Substring Without Repeating Characters	Medium	⚡ MLSys Connection: Sliding window is the standard pattern for contiguous sub-array problems. It is widely used in network protocol layers (e.g. TCP sliding window congestion control) and stream processing systems (e.g. computing average request rates over a sliding temporal window).
			Key Points: Sliding Window + Freq Tracking
424	Longest Repeating Character Replacement	Medium	⚡ MLSys Connection: The window condition is that window length L minus the frequency of the most frequent character <code>max_freq</code> must be $\leq k$. We maintain a historical <code>max_freq</code> to avoid scanning the frequency map of size 26 on every window contraction.
			Key Points: Sliding Window + Dual Hash Maps
76	Minimum Window Substring	Hard	⚡ MLSys Connection: A benchmark problem for hard sliding windows. In compiler lexical analyzers and low-level pattern scanners, similar sliding window

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			logic is used for high-performance syntax token parsing.

Day 03 Fast & Slow Pointers

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			Key Points: Floyd's Cycle-Finding Algorithm
141	Linked List Cycle	Easy	⚡ MLSys Connection: Fast and slow pointers are used in OS memory managers to detect if free memory blocks or page descriptors have circular pointer references (which causes memory leak traps or segmentation faults) due to corrupt allocations.
			Key Points: Floyd's Algorithm for Loop Entry
142	Linked List Cycle II	Medium	⚡ MLSys Connection: Mathematical proof shows that the distance from the head to the loop entry equals the distance from the pointer meeting point to the loop entry. This demonstrates precise pointer distance manipulation.
			Key Points: Implicit Linked List Cycle Detection
202	Happy Number	Easy	⚡ MLSys Connection: Transformations of a number can be modeled as a directed graph where nodes have out-degree 1 (analogous to a singly linked list). Circular dependency detection can be solved using fast and slow pointers in $O(1)$ auxiliary space.

Day 04

Monotonic Stack

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
496	Next Greater Element I	Easy	Key Points: Monotonic Decreasing Stack ⚡ MLSys Connection: Monotonic stacks are optimal for 'finding the next greater element'. By maintaining stack monotonicity, we guarantee each element is pushed and popped at most once, achieving a linear $O(N)$ time complexity.
739	Daily Temperatures	Medium	Key Points: Monotonic Stack Storing Indices ⚡ MLSys Connection: In time-series analysis and system latency metric telemetry (e.g. finding the next timestamp where queue latency drops below a threshold), monotonic stack-based index tracking is a fundamental building block.
84	Largest Rectangle in Histogram	Hard	Key Points: Monotonic Increasing Stack + Sentinels ⚡ MLSys Connection: Used in memory defragmentation (e.g. finding the largest contiguous blocks of physical memory pages). The algorithm calculates areas from the popped taller blocks in $O(N)$ time.

Day 05

Monotonic Queue

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
239	Sliding Window Maximum	Hard	Key Points: Monotonic Queue (Double-Ended Queue)

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			⚡ MLSys Connection: In MLSys inference servers, we must compute sliding window metrics such as peak inference latency or throughput. Using a double-ended queue (deque) to maintain monotonicity ensures $O(N)$ time complexity for real-time monitoring statistics.
862	Shortest Subarray with Sum at Least K	Hard	Key Points: Prefix Sum + Monotonic Deque ⚡ MLSys Connection: Finding the shortest subarray satisfying an accumulation threshold in data streams containing negative values. Combining prefix sums with a monotonic deque achieves $O(N)$ optimal search time.

Day 06

Prefix Sums & Difference Arrays

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
304	Range Sum Query 2D - Immutable	Medium	Key Points: 2D Prefix Sums / Integral Image ⚡ MLSys Connection: A 2D prefix sum is mathematically identical to an 'Integral Image' in computer vision. In CNN (Convolutional Neural Networks) pooling layers or local sum kernels, it optimizes range sum calculations from $O(H*W)$ to $O(1)$.
1109	Corporate Flight Bookings	Medium	Key Points: 1D Difference Array ⚡ MLSys Connection: Difference arrays are used to handle batch range addition/subtraction. In task scheduling, if we need to allocate resources over a time

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			window, difference arrays allow $O(1)$ updates per request and reconstruction in $O(N)$ time via prefix sums.
1094	Car Pooling	Medium	Key Points: Difference Array Overload Checks ⚡ MLSys Connection: Used in distributed system concurrency throttling and dynamic GPU memory budget tracking. We convert intervals of resource demands into difference arrays to verify if peak load violates hard hardware capacity constraints.

Day 07 Matrix Operations & Locality

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
48	Rotate Image	Medium	Key Points: In-Place Transpose + Reflection ⚡ MLSys Connection: Matrix transpose and reflection inspect Cache Locality. In MLSys, tensor dimensions permutes (Transpose/Permute) are extremely common. Fast strided addressing and in-place swapping on contiguous 1D memory buffers are central to GPU kernel optimization.
54	Spiral Matrix	Medium	Key Points: Boundary Pointer Shrinking Simulation ⚡ MLSys Connection: Clockwise spiral traversal. It requires strict boundary checks (top, bottom, left, right) to prevent out-of-bounds access, testing precision in array slicing and boundary control.

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
73	Set Matrix Zeroes	Medium	Key Points: Row/Column Projection Markers ⚡ MLSys Connection: An in-place marking scheme when memory is extremely constrained. By projecting zero states onto the first row and first column of the matrix, we achieve $O(1)$ auxiliary space, which is highly practical in embedded or edge computing kernels.

Phase 2: Trees & Binary Search (Days 8-14)

Phase Target: Avoid infinite loops in binary search boundary updates, master recursive derivations of tree/BST structures, and build confidence in backtracking and state space pruning.

Day 08

Binary Search

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
704	Binary Search	Easy	Key Points: Standard Binary Search Template ⚡ MLSys Connection: The essence of binary search is partition boundary definition and loop condition control. In low-level systems, such as OS memory page table lookup or indexing in sorted key-value databases, binary search is the default $O(\log N)$ lookup algorithm.
33	Search in Rotated Sorted Array	Medium	Key Points: Monotonic Half Partitioning ⚡ MLSys Connection: Search in rotated arrays. If split in half, at least one half of the rotated sorted array is guaranteed to be strictly sorted. Detecting which half is sorted allows us to halve the search space at each step.
153	Find Minimum in Rotated Sorted Array	Medium	Key Points: Inflection Point Binary Search ⚡ MLSys Connection: Compare <code>nums[mid]</code> with the right boundary <code>nums[right]</code>. If <code>nums[mid] > nums[right]</code>, the inflection point is in the right half, otherwise in the left. This tests the ability to leverage monotonicity in non-trivially sorted sequences.

Day 09 Advanced Binary Search

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
74	Search a 2D Matrix	Medium	Key Points: Matrix 1D Flat Indexing ⚡ MLSys Connection: A row-major sorted matrix is stored contiguously in memory as a 1D array. By mapping indices via <code>(row, col) = (mid // n, mid % n)</code>, we run standard binary search directly on the flat memory block, avoiding conversion overhead.
162	Find Peak Element	Medium	Key Points: Hill Climbing Binary Search ⚡ MLSys Connection: In optimization theory and adaptive parameter tuning, 'Hill Climbing' is the fundamental local search heuristic. If the slope at <code>mid</code> is ascending, the peak must lie to the right, which guarantees locating a peak in $O(\log N)$ time.
4	Median of Two Sorted Arrays	Hard	Key Points: Binary Search on Array Partitioning ⚡ MLSys Connection: In distributed databases or multi-GPU pipeline parallelisms, we often need to merge and find the median of sorted, partitioned chunks. Performing binary search on partition lines in the smaller array resolves this in $O(\log(\min(M, N)))$ time.

Day 10 Tree Recursion & BFS

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
104	Maximum Depth of Binary Tree	Easy	Key Points: DFS Post-Order Recursion ⚡ MLSys Connection: The recursion depth of a binary tree determines the call stack depth. When writing AST (Abstract Syntax Tree) parsers or computational graph execution engines, we must account for call stack limits (worst-case stack overflow if the tree degenerates into a list of size N).
226	Invert Binary Tree	Easy	Key Points: In-Place Subtree Swapping ⚡ MLSys Connection: A classic structural transformation. It tests fundamental binary tree recursion, return values handling, and in-place child pointer swaps.
102	Binary Tree Level Order Traversal	Medium	Key Points: BFS Level-by-Level Queue ⚡ MLSys Connection: Level order traversal is the basis of BFS. In graph execution engines and distributed schedulers (e.g. Ray executing dataflow tasks by dependency layer), processing nodes level-by-level ensures strict causal order.

Day 11

BST & LCA

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
98	Validate Binary Search Tree	Medium	Key Points: BST Bounds Propagation ⚡ MLSys Connection: BST validation is a global property. We must pass valid open interval ranges `(lower, upper)` down to children during recursion. Simply checking

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			if local child nodes satisfy local invariants is a common trap.
235	Lowest Common Ancestor of a Binary Search Tree	Medium	Key Points: BST Path Partitioning ⚡ MLSys Connection: If both targets are smaller than current, go left; if larger, go right; else the paths split and current is the LCA. Leveraging BST ordering yields an O(H) time and O(1) space optimal solution.
236	Lowest Common Ancestor of a Binary Tree	Medium	Key Points: General Binary Tree Post-Order DFS ⚡ MLSys Connection: General trees do not have value ordering, requiring a post-order traversal to bubble up match flags. If left and right subtrees both return non-null, the current node is the LCA.

Day 12

Tree Construction & DFS

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
105	Construct Binary Tree from Preorder and Inorder Traversal	Medium	Key Points: Pre/In-Order Partition + Hash Mapping ⚡ MLSys Connection: In compiler parsers, building an AST from a token sequence and in-order grammar rules is a core compiler task. Caching in-order indices in a hash map optimizes reconstruction time complexity from $O(N^2)$ to $O(N)$.
437	Path Sum III	Medium	Key Points: DFS + Prefix Sum Hash Map ⚡ MLSys Connection: Porting prefix sum logic to trees. During DFS, we maintain

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			running prefix sums in a hash map, backtracking to remove current paths when returning. This yields an O(N) time and O(H) space solution.
124	Binary Tree Maximum Path Sum	Hard	Key Points: DFS + Bent Path Global Update ⚡ MLSys Connection: A classic tree dynamic programming problem. In Critical Path Method (CPM) analyses of computation graphs (determining which sequence of operators is the bottlenecks), this path accumulation logic is used to identify the slowest execution chain.

Day 13

Backtracking Basics

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
78	Subsets	Medium	Key Points: Backtracking / Bitmasking ⚡ MLSys Connection: Generating subsets is equivalent to traversing a decision tree. In AI compiler operator fusion search space, we evaluate all subset combinations of nodes to find the best kernel fusion plan. Search space size is 2^N .
46	Permutations	Medium	Key Points: Decision Tree + Visited Array ⚡ MLSys Connection: Permutations represent task scheduling setups. Schedulers run permutations to evaluate all potential task execution orders for instruction-level scheduling optimizations.

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
39	Combination Sum	Medium	Key Points: Backtracking + Index Deduplication ⚡ MLSys Connection: Solving combination sums. By passing a `start` search index, we avoid duplicate combinations, demonstrating the basic backtracking constraint patterns.

Day 14 Advanced Backtracking

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
51	N-Queens	Hard	Key Points: Backtracking + Diagonal Hash Tables ⚡ MLSys Connection: Classic N-Queens backtracking. The key optimization is using arrays for `r - c` and `r + c` to identify diagonals, enabling O(1) conflict validation instead of O(N) board scans.
212	Word Search II	Hard	Key Points: Trie + Grid DFS Backtracking + Pruning ⚡ MLSys Connection: Build a Trie from target words. Walk the grid and Trie pointers simultaneously. To pass tight limits, when a word is matched, we remove it from the Trie, and prune empty subtrees in-place, eliminating redundant traversal paths.

Phase 3: Graphs & Systems Data Structures (Days 15-21)

Phase Target: Master topological sort for task scheduling, Union-Find for connectivity, Trie prefixes, heap scheduling, and hand-code high-performance concurrent Cache systems (LRU/LFU).

Day 15 Graph Traversals

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
200	Number of Islands	Medium	Key Points: Grid BFS / DFS Connectivity ⚡ MLSys Connection: In MLSys tensor computational graph partitioning, splitting massive execution graphs into independent subgraphs for distributed training is identical to finding connected island components. In-place modification is used to save memory.
133	Clone Graph	Medium	Key Points: Hash Map + DFS / BFS Deep Copy ⚡ MLSys Connection: In deep learning compiler IR optimizations or computational graph serialization, we must copy graphs containing cycles (e.g. recurrent neural networks). Maintaining an `old_node -> new_node` hash map prevents infinite recursion loop traps.
417	Pacific Atlantic Water Flow	Medium	Key Points: Multi-Source Reverse DFS from Boundaries ⚡ MLSys Connection: Multi-source reverse search. Instead of searching down to ocean boundaries from each cell (which times out), we search uphill from the boundaries. This yields a clean $O(M*N)$ marking of double-reachability.

Day 16

Topological Sort & Scheduling

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
207	Course Schedule	Medium	Key Points: Kahn's BFS In-Degree Algorithm ⚡ MLSys Connection: Modern ML frameworks (PyTorch, TensorFlow) structure execution DAGs (Directed Acyclic Graphs). Prerequisite dependencies represent edges. The compiler schedules operator executions in topological order to satisfy dependency constraints.
210	Course Schedule II	Medium	Key Points: Topological Sequence Output ⚡ MLSys Connection: Similar to 207, but collects dequeued nodes. If the sequence length matches the total nodes, the graph is a DAG, return the sorted list, otherwise return empty due to circular deadlock dependencies.
269	Alien Dictionary	Hard	Key Points: DFS Cycle Detection + Reverse Finish Order ⚡ MLSys Connection: In deep learning compiler instruction schedulers, we infer instruction dependency constraints from adjacent operations. Constructing directed edges and running a topological sort resolves execution priorities.

Day 17

Union-Find & Equivalence Classes

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
261	Graph Valid Tree	Medium	Key Points: Union-Find Cycle and Connectivity Checks

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			⚡ MLSys Connection: Union-Find is the optimal structure for grouping disjoint equivalence classes. An undirected graph is a tree if edges = n-1 and Union-Find finds no cycles (unioning two nodes already in the same root set <code>find(u) == find(v)</code> yields a cycle).
323	Number of Connected Components in an Undirected Graph	Medium	Key Points: Union-Find Component Counter ⚡ MLSys Connection: Initialize component count to n. For each successful merge of disjoint sets, decrement count by 1. Union-Find with path compression and rank-based unioning executes in near-constant O(1) amortized time per operation.
684	Redundant Connection	Medium	Key Points: Union-Find Cycle Pinpointing ⚡ MLSys Connection: In redundant network routing path pruning, we locate the final edge that creates a loop. Processing edges sequentially with Union-Find identifies the first cycle-creating edge in O(E) time.

Day 18

Prefix Trees (Trie)

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
208	Implement Trie (Prefix Tree)	Medium	Key Points: Nested Hash Map Children Nodes ⚡ MLSys Connection: In Large Language Model (LLM) speculative decoding, serving engines use Tries to index vocabularies, enabling fast prefix constraints validation

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			and KV Cache token matches, drastically reducing sequence comparisons.
211	Design Add and Search Words Data Structure	Medium	Key Points: Trie Wildcard Search via DFS ⚡ MLSys Connection: Extends standard Trie search with wildcard '.' handling. When encountering '.', we recursively check all child branches of the current node, demonstrating recursive Trie search patterns.
642	Design Search Autocomplete System	Hard	Key Points: Trie + Min-Heap Top-K Filter ⚡ MLSys Connection: Simulating search engine auto-complete. In speculative decoding or prefix tuning, filtering high-frequency predictions based on string prefixes represents a critical performance path.

Day 19

Heaps & Priority Schedulers

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
23	Merge k Sorted Lists	Hard	Key Points: Min-Heap Node Tracking / Divide & Conquer ⚡ MLSys Connection: In distributed stream mergers or log consolidation engines, a min-heap tracks the current head of k sorted streams. This runs in $O(N \log k)$ time and represents a core scheduling technique in database log mergers.
347	Top K Frequent Elements	Medium	Key Points: Hash Map + Bucket Sort ($O(N)$ Optimal)

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			⚡ MLSys Connection: While a min-heap gives $O(N \log K)$, the optimal solution uses bucket sort. Because frequencies are bounded by $[0, N]$, bucket sorting allows linear $O(N)$ retrieval, highlighting a data-driven system optimization choice.
295	Find Median from Data Stream	Hard	Key Points: Dual Balanced Heaps (Max & Min) ⚡ MLSys Connection: In MLSys runtime telemetry modules, we must compute the median inference latency in $O(1)$ time while accepting new data points in $O(\log N)$ time. This is solved by balancing a Max-Heap (smaller values) and a Min-Heap (larger values).

Day 20

Bit Manipulation & Lower-Precision Formats

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
136	Single Number	Easy	Key Points: Bitwise XOR properties ($a \oplus a = 0$) ⚡ MLSys Connection: XOR properties ($x \oplus x = 0$, $x \oplus 0 = x$) and associativity allow $O(N)$ scan in $O(1)$ space. This is a common parity check and data validation technique in hardware-constrained embedded applications.
338	Counting Bits	Easy	Key Points: DP Bitwise Recurrence ⚡ MLSys Connection: Using $dp[i] = dp[i \gg 1] + (i \& 1)$ computes set bits for all numbers from 0 to N in $O(N)$ time. Used in quantized weights bitwidth profiling for

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			Binary Neural Networks (BNNs) and INT8 model quantization.
190	Reverse Bits	Easy	Key Points: Bitwise Shifts & Accumulation ⚡ MLSys Connection: Bitwise swaps. Required for network packet conversions (Big-endian to Little-endian byte order transformations) and graphics pixel bit alignment.
137	Single Number II	Medium	Key Points: Bit-position Modulo Arithmetic ⚡ MLSys Connection: Simulating logic gates in hardware accelerators (ASICs/FPGAs) or SIMD vector optimizations to track duplicate cycles without using general registers.

Day 21

Cache System Implementation

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
146	LRU Cache	Medium	Key Points: Hash Map + Doubly Linked List ($O(1)$) ⚡ MLSys Connection: A primary MLSys design challenge. LRU page eviction is the default mechanism for LLM KV Cache paging (e.g. vLLM's PagedAttention) and deep learning Parameter Server weight swapping. You must hand-code it in $O(1)$ time using a hash map and a doubly linked list. Often extended to discuss thread safety via shared reader-writer locks (shared_mutex) or fine-grained bucket locks in concurrent implementations.

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
460	LFU Cache	Hard	Key Points: Double Hash Maps + Frequency Lists ⚡ MLSys Connection: Hard cache eviction. Evicts the least frequently used node, breaking ties using LRU. Requires maintaining a <code>min_freq</code> integer and a mapping of frequencies to doubly linked lists, challenging data structure composition skills.
622	Design Circular Queue	Medium	Key Points: Circular Array Modulo Arithmetic ⚡ MLSys Connection: Ring buffers are the standard data structures for OS IPC (Inter-Process Communication), producer-consumer loops, and GPU command queues. Modulo index updates <code>(tail + 1) % size</code> enable lock-free concurrent ring buffer queues.

Phase 4: Dynamic Programming & High-Pressure Mock (Days 22-30)

Phase Target: Master 1D/2D state transitions, apply space compression optimizations, and practice solving unseen tasks under strict time-box constraints.

Day 22

1D Dynamic Programming

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
70	Climbing Stairs	Easy	Key Points: 1D Recurrence + Space Compression ⚡ MLSys Connection: Basic dynamic programming. Rolling variables <code>a, b = b, a + b</code> compress space from $O(N)$ to $O(1)$. In MLSys tensor compilers, minimizing memory footprint via buffer reuse is a key optimization goal.
746	Min Cost Climbing Stairs	Easy	Key Points: Weighted 1D DP Recurrence ⚡ MLSys Connection: DP recurrence <code>dp[i] = cost[i] + min(dp[i-1], dp[i-2])</code>. Can be optimized to $O(1)$ space, helping to build basic dynamic programming muscle memory.
322	Coin Change	Medium	Key Points: Unbounded Knapsack - Bottom-Up DP ⚡ MLSys Connection: Optimization problem where greedy choices fail. Transition equation: <code>dp[i] = min(dp[i], dp[i - coin] + 1)</code>. In runtime system resource scheduler budget allocations, knapsack allocation models are widely applied.

Day 23

Subsequences & Knapsack DP

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
300	Longest Increasing Subsequence	Medium	Key Points: Patience Sorting / Greedy + Binary Search ⚡ MLSys Connection: Optimal $O(N \log N)$ using greedy tail replacements and binary search. In compiler instruction pipelining, identifying longest non-dependent instruction sequences dictates parallel execution schedules.
139	Word Break	Medium	Key Points: Prefix Partitioning DP + Hash Set ⚡ MLSys Connection: Evaluating segmentations of contiguous streams. Transition: <code>`dp[i] = any(dp[j] and s[j:i] in wordSet for j in range(i))`</code>. In compiler lexical parsing and token segmentations, prefix partitioning plays a central role.
416	Partition Equal Subset Sum	Medium	Key Points: 0-1 Knapsack with Space Compression ⚡ MLSys Connection: Classic 0-1 knapsack. To compress space, we update a 1D DP array backwards (from right to left) to prevent using values updated in the current iteration, dropping space from $O(\text{target})$ to $O(\text{target})$.

Day 24

2D Grid Dynamic Programming

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
62	Unique Paths	Medium	Key Points: Grid DP + Row Rolling Compression ⚡ MLSys Connection: Grid routing. Transition: <code>dp[i][j] = dp[i-1][j] + dp[i][j-1]</code>. In Network-on-Chip (NoC) hardware routing designs, analyzing packet routing paths matches this model. Space can be compressed to a 1D row array.
1143	Longest Common Subsequence	Medium	Key Points: 2D Alignment DP Matrix ⚡ MLSys Connection: LCS is the core algorithm for compiler code-diff checkers and structural network similarity evaluations. Time complexity is $O(M*N)$, and rolling row array updates are often asked in system memory optimization rounds.
72	Edit Distance	Hard	Key Points: 2D Match Decision Matrix ⚡ MLSys Connection: Minimum edit cost calculation. Analyzing state transitions from three cells <code>dp[i-1][j]</code>, <code>dp[i][j-1]</code>, and <code>dp[i-1][j-1]</code> evaluates comprehensive 2D dynamic programming logic.

Day 25

Interval & Tree DP

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
96	Unique Binary Search Trees	Medium	Key Points: Catalan Number Recurrence ⚡ MLSys Connection: Counting possible BST shapes. Summing the product of left and right child permutations derives the Catalan recurrence, showcasing algebraic state symmetry.
337	House Robber III	Medium	Key Points: Tree DP + State Tuple DFS Propagation ⚡ MLSys Connection: In computational graph kernel fusions, selecting optimal operator combinations where adjacent nodes cannot be fused matches tree DP. DFS returns a tuple `(fuse, skip)`, constraining time complexity to O(N).
312	Burst Balloons	Hard	Key Points: Interval DP (Bottom-Up) ⚡ MLSys Connection: Advanced interval DP. Reversing the problem to identify 'which balloon is popped last' separates the interval into independent segments, defining the state transition. Crucial for complex compiler scheduler optimizations.

Day 26

Greedy Algorithms

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
55	Jump Game	Medium	Key Points: Greedy - Backwards Goal Shifting ⚡ MLSys Connection: Evaluating if a resource packet can route across nodes with variable reach capabilities. Shifting `goal` backwards from the end runs in O(N)

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
			time and $O(1)$ space, which is far superior to standard 1D DP ($O(N^2)$).
134	Gas Station	Medium	Key Points: Net Sum Check + Greedy Pivot Search ⚡ MLSys Connection: If total supply exceeds total consumption, a solution must exist. A single scan shifts the starting candidate to $i + 1$ whenever the rolling sum drops below 0, optimizing the $O(N^2)$ search to $O(N)$.
406	Queue Reconstruction by Height	Medium	Key Points: Greedy Insertion after Multi-Key Sort ⚡ MLSys Connection: Scheduling multi-resource allocations with two priorities. Sorting height descending and queue index ascending, then executing index insertions, yields the local queue reconstruction.

Day 27

Intervals & Scheduling

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
56	Merge Intervals	Medium	Key Points: Sort by Start Time + Linear Merge ⚡ MLSys Connection: In OS virtual memory managers, when pages are deallocated, contiguous free blocks are coalesced (merged) to minimize memory fragmentation. Coalescing free blocks is logically equivalent to interval merging.

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
435	Non-overlapping Intervals	Medium	Key Points: Greedy - Sort by End Time ⚡ MLSys Connection: Classic interval scheduling. To maximize scheduled tasks, prioritize intervals that end earliest. Used in real-time priority task scheduling and instruction pipelines.
621	Task Scheduler	Medium	Key Points: Greedy Bucket Fill with Cooldowns ⚡ MLSys Connection: In CPU schedulers or GPU thread dispatchers, tasks of the same type must wait for a Cooldown cycle. Estimating cycles by filling buckets with high-frequency tasks resolves the scheduling pattern in O(N) time.

Day 28

Mock Simulator Day 1 (Google/Meta Core Systems)

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
307	Range Sum Query - Mutable	Hard	Key Points: Segment Tree / Binary Indexed Tree ⚡ MLSys Connection: In dynamic ML tensors where values are frequently mutated and range sums are queried, prefix sums cost O(N) updates. Segment Trees or Fenwick Trees optimize both operations to O(log N) time, demonstrating query-update balance.
588	Design In-Memory File System	Hard	Key Points: Trie with File Node Directory Tree ⚡ MLSys Connection: Designing an OS in-memory file system. Models a directory hierarchy using a Trie where terminal nodes

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
----	---------	------------	--

represent file contents, supporting ``ls``, ``mkdir``, ``addContentToFile``, and ``readContentFromFile`` operations.

Day 29

Mock Simulator Day 2 (NVIDIA/Apple Hardware & Concurrency)

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
----	---------	------------	--

311	Sparse Matrix Multiplication	Medium	Key Points: Compressed Row Storage (CRS / CSR) Optimization ⚡ MLSys Connection: The core kernel of Sparse Neural Networks. In GPU computations, performing dense multiplications on massive zero arrays wastes memory bandwidth. Iterating only over non-zero elements using Row-major formats simulates CSR acceleration hardware kernels.
-----	--	--------	---

380	Insert Delete GetRandom O(1)	Medium	Key Points: Hash Map + Dynamic Array Tail Swap ⚡ MLSys Connection: Used in MLSys negative batch samplers requiring $O(1)$ random draws, inserts, and deletes. A map indexes keys to list offsets. To delete in $O(1)$ time, swap the target element with the last element of the list, then pop back to avoid array shifting.
-----	--	--------	---

Day 30

Mock Simulator Day 3 (FAANG Hard Graduation Challenges)

ID	Problem	Difficulty	Key Points & MLSys Low-Level Systems Mapping
460	LFU Cache	Hard	Key Points: Hand-Code LFU (Review Day 21) ⚡ MLSys Connection: High-performance cache design under pressure. Evict least-frequently-used nodes, requiring double hash maps and frequency doubly linked list updates in O(1) time.
212	Word Search II	Hard	Key Points: Hand-Code Trie + Backtracking Grid DFS (Review Day 14) ⚡ MLSys Connection: Integrates Tries, matrix traversals, backtracking, and recursive parent pruning optimizations. Evaluates graph searches and pruning under tight limits.
269	Alien Dictionary	Hard	Key Points: Hand-Code Topological Sort & DFS Cycle Detect (Review Day 16) ⚡ MLSys Connection: The foundation of compilation pipeline dependencies scheduling. Combines parsing, directed graph construction, cycle validation, and topological sorting.